

SCHOOL OF ELECTRONICS ENGINEERING

WINTER SEMESTER 2025-26

Embedded C Programming

Project Report

Motor torque control using battery stress aware system in EVs

Name: S Venkatesan

Reg no: 23BEC0368

Slot: B2

Submitted to : Dr Sumathi G

Team mates:

Shree Hari D

(23BEC0355)

V Rithika

(23BEC0028)

TABLE OF CONTENTS

1.Abstract

2.Literature Survey

3.Problem Statement

4.Proposed Methodology
- 4.1 System Overview
- 4.2 Mathematical Model (Battery Stress Equation)
- 4.3 Control Strategy (Normal & Emergency Mode)

5.System Design
- 5.1 Flow Chart
- 5.2 Block Diagram
- 5.3 hard ware/software used

6.Results and Analysis
- 6.1 Experimental Results
- 6.2 Performance Analysis
- 6.3 Stress vs PWM Behavior

7.Comparative Study with Existing Systems

8.Project Timeline

9.Cost Analysis

10.Future Work

11.References

12.Appendix

1.ABSTRACT

Electric vehicles demand intelligent control strategies to ensure both performance and battery longevity. This project presents a real-time motor control system that adapts motor speed based on battery stress conditions. The system continuously monitors key parameters such as current, temperature, and battery State of Charge (SOC), and combines them into a nonlinear Battery Stress Index (BSI).
An ESP32 microcontroller processes sensor input and dynamically adjusts motor torque using PWM control. Unlike conventional systems that rely on fixed thresholds or complex predictive models, the proposed approach uses a lightweight mathematical model suitable for embedded implementation.
The system also introduces dual operating modes: a normal mode that prioritizes battery safety, and an emergency mode that prioritizes performance while maintaining basic protection. Additionally, a low-temperature compensation mechanism is incorporated to maintain drivability under cold conditions.
The overall system achieves smooth motor control, improved battery protection, and cost-effective implementation, making it suitable for practical and scalable electric vehicle applications.

2. Literature Survey

Recent developments in electric vehicle (EV) systems emphasize the importance of regulating motor torque based on battery conditions to ensure safety, efficiency, and longevity. Various industrial patents and research studies have addressed this challenge using different control strategies.

Tesla's approach [1] focuses on estimating the maximum discharge power of the battery using real-time measurements of State of Charge (SOC) and temperature. The motor torque is then limited accordingly to prevent excessive battery stress. While effective, this method primarily relies on power clipping rather than adaptive multi-parameter control.

Ford's method [2] introduces temperature-dependent torque derating using predefined lookup tables. Although it provides protection against thermal overload, the approach lacks flexibility and does not adapt dynamically to rapid changes in operating conditions.

General Motors proposed a Model Predictive Control (MPC)-based strategy [3], where future battery states are predicted to optimize torque delivery. This method offers high accuracy but requires complex mathematical modeling and significant computational resources, making it less suitable for low-cost embedded systems.

Toyota's work [4] addresses the impact of low temperatures on battery performance. It highlights how increased internal resistance under cold conditions reduces the battery's ability to deliver power, leading to torque limitations. However, the approach is primarily focused on specific environmental conditions and does not integrate multiple stress factors into a unified framework.

Torque arbitration techniques [5] introduce a centralized control mechanism that determines safe torque limits based on multiple constraints. Similarly, State of Power (SOP) estimation methods [6] evaluate the maximum deliverable power of a battery using voltage, current, and temperature. These approaches improve safety but often depend on complex system-level integration.

In academic research, Moura et al. [7] presented a battery-constrained optimal control framework that balances motor performance with battery health using optimization techniques. Sun et al. [8] proposed a multi-constraint SOP estimation method that accurately predicts peak power capability under varying conditions. Furthermore, Singh and Williamson [9] provided a comprehensive review of battery management systems, comparing various control techniques including fuzzy logic, neural networks, and rule-based methods.

Despite these advancements, most existing systems either rely on computationally intensive models or fixed threshold-based control strategies. This limits their applicability in simple, real-time embedded implementations.

To address these limitations, the proposed system introduces a nonlinear battery stress-based control approach that integrates current, temperature, SOC, and dynamic current variation into a unified stress index. This enables real-time adaptive motor control using low-cost hardware, bridging the gap between advanced EV control systems and practical embedded solutions.

3. PROBLEM STATEMENT

Battery degradation and inefficient motor control are major challenges in electric vehicles. Excessive current draw, high temperature, and low SOC can lead to reduced battery life and unsafe operation.
Existing solutions either:
Use fixed thresholds (not adaptive), or
Use complex predictive algorithms (not practical for small systems)
Hence, there is a need for a system that:
Monitors battery condition in real time
Adapts motor speed dynamically
Maintains smooth driving experience
Works efficiently on low-cost hardware

4. Proposed Methodology

4.1 System Overview

The proposed system implements an intelligent motor control strategy based on real-time battery condition monitoring. The system continuously acquires data from current, temperature, and voltage sensors to evaluate battery health and dynamically adjust motor performance.

An ESP32 microcontroller acts as the central processing unit, where all sensor data is processed to compute a battery stress index. Based on this stress value and user input (mode selection), the system regulates motor speed using PWM control.

Two operating modes are implemented:

Normal Mode: Prioritizes battery safety and longevity by reducing motor speed under high stress conditions.

Emergency Mode: Allows higher motor output with minimal restrictions, ensuring performance when required while still maintaining basic safety limits.

Additionally, the system includes low-temperature handling, where performance is naturally limited due to battery inefficiency, and control logic ensures smooth operation without instability.

4.2 Mathematical Model (Battery Stress Equation)

To overcome limitations of fixed threshold-based systems, a nonlinear stress model is developed.

The Battery Stress Index (BSI) is calculated as:

$$BSI = w_1 (I/I_{max})^2 + w_2 (T/T_{max})^3 + w_3 1\text{-}SOC) + w_4 \, |dI/dt|$$

Where:

I = Battery current

T = Temperature

SOC = State of Charge

dI/dt = Rate of change of current

w1,w2,w3,w4 = Weighting factors

Enhancements implemented:

Nonlinear terms (square, cube) to capture real battery behavior

Inclusion of dynamic load variation (dI/dt)

Real-time computation on ESP32

Low-pass filtering applied to reduce noise and stabilize output

This model provides a more accurate and responsive estimation of battery stress compared to conventional lookup tables.

4.3 Control Strategy (Normal & Emergency Mode)

Based on the computed stress value, motor control is implemented as follows:

Normal Mode (Battery Protection Mode)

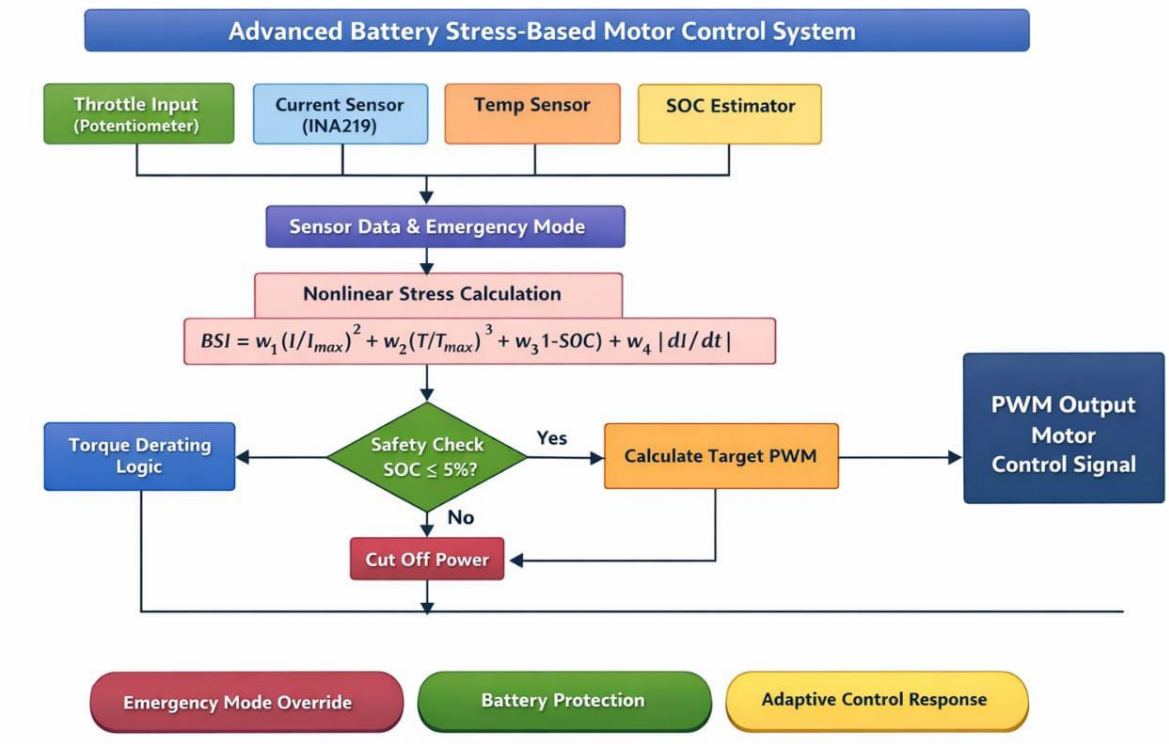
If stress < 0.3 → Full speed (no restriction)

If 0.3 ≤ stress < 0.6 → Moderate reduction (~80%)

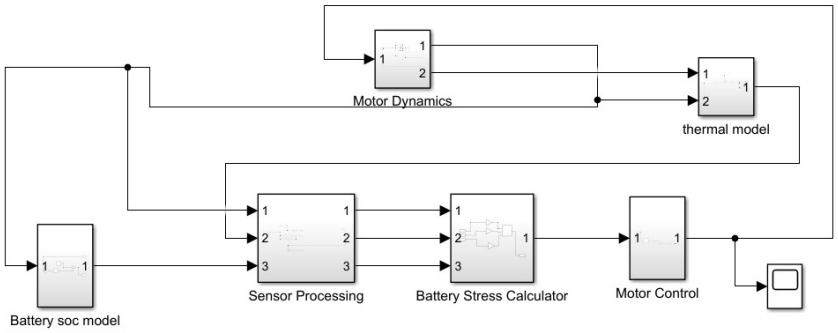
If stress ≥ 0.6 → Strong reduction (~60%)
This ensures gradual performance reduction and avoids sudden motor behavior.
Emergency Mode (Performance Mode)
Motor operates at ~95% of throttle input
Only extreme stress (>0.85) triggers slight reduction
This mode ensures:
Better drivability
Immediate response
Controlled safety override
Additional Safety Mechanisms
SOC Cutoff: Motor stops when SOC ≤ 5%
Torque Derating: Current-based scaling to prevent overload
Smooth PWM Transition: Prevents jerks
Filtering: Stabilizes temperature and current readings

5.System design:

5.1 Flow chart :



5.2 Block Diagram



5.3 Hardware/software Used

Hardware used:

ESP32
INA219 Current Sensor
Temperature Sensor (LM35)
Voltage Sensor Module
Motor Driver
DC Motor
Battery
Push Button

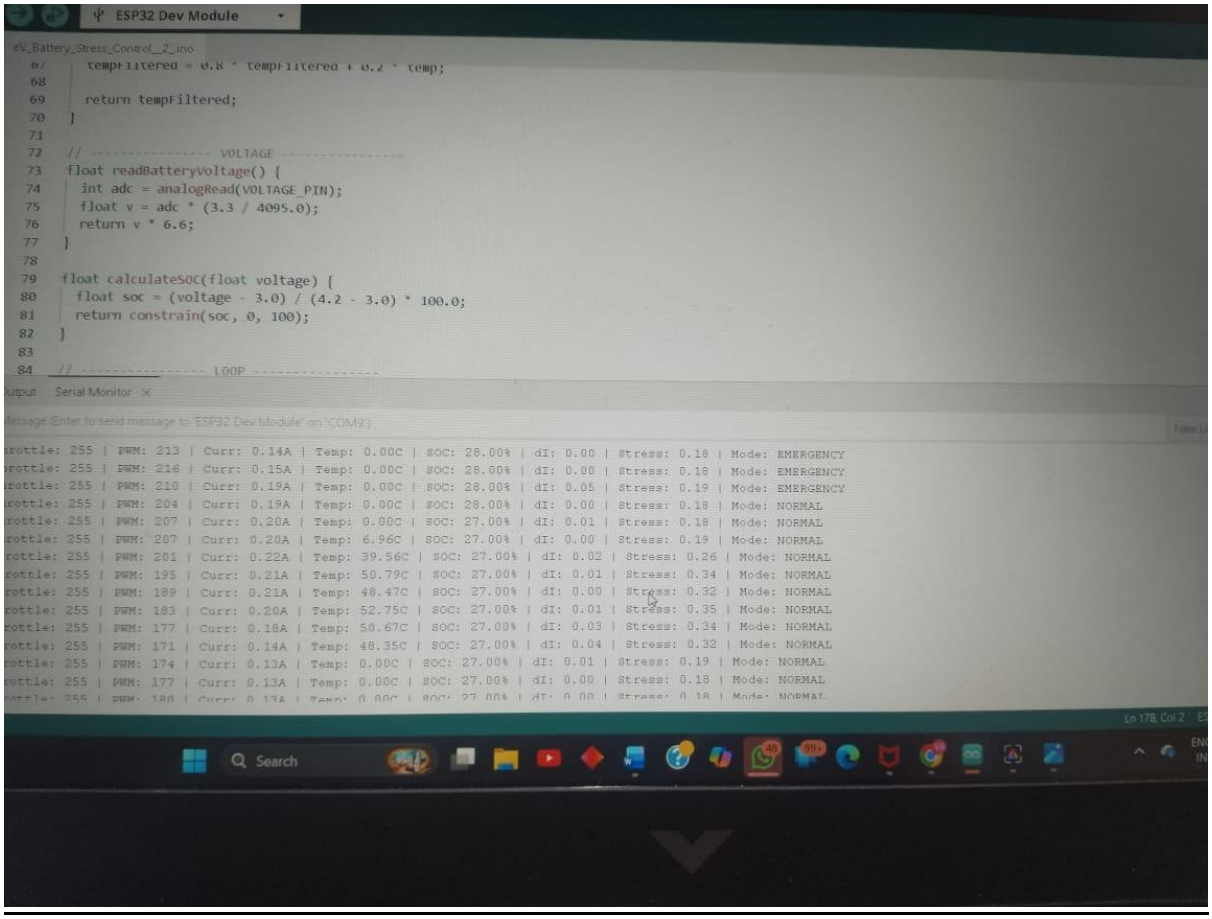
Software Used:

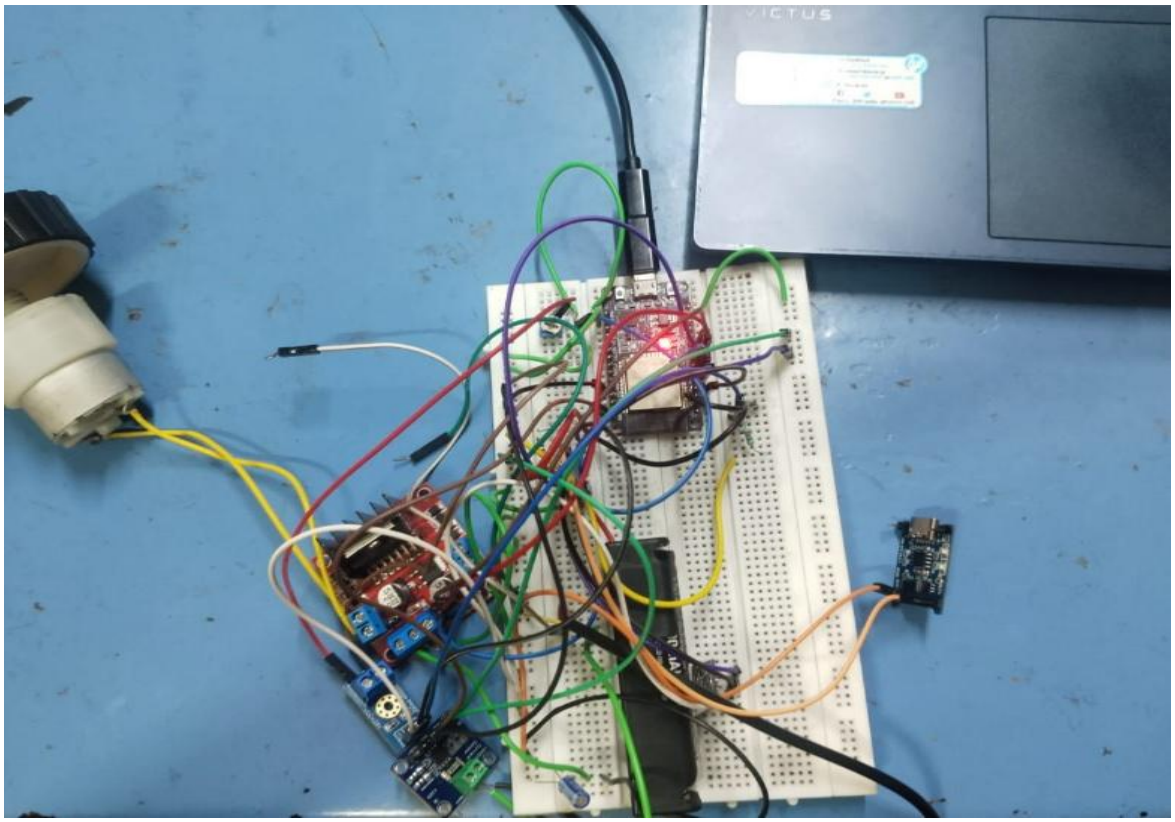
Arduino IDE
Embedded C

6. Results and Analysis

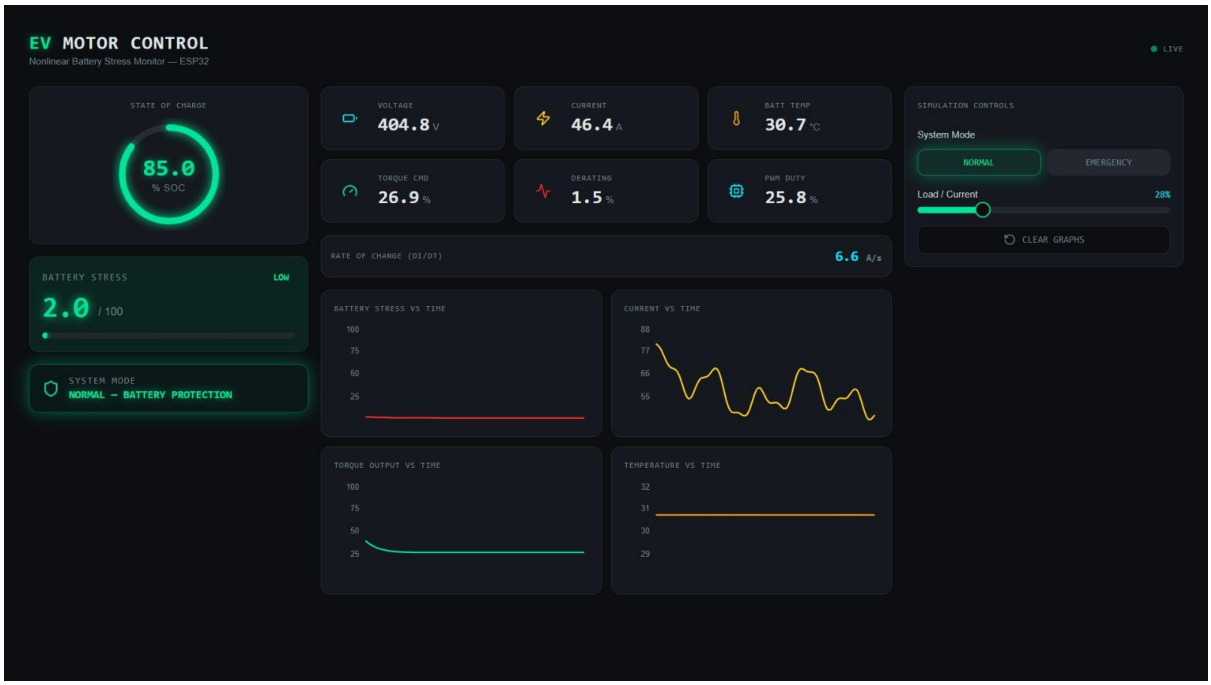
6.1 Experimental Results

The system was tested under different operating conditions by varying throttle input, temperature, and load.
The following observations were recorded:
Real-time display of:
Current (INA219)
Temperature
Simulated SOC
Real SOC (from voltage)
Stress value
PWM output
:


ESP32 Dev Module
eV_Battery_Stress_Control_2.ino
tempFiltered = 0.8 * tempFiltered + 0.2 * temp;
return tempFiltered;
// ---------- VOLTAGE ----------
float readBatteryVoltage() {
int adc = analogRead(VOLTAGE_PIN);
float v = adc * (3.3 / 4095.0);
return v * 6.6;
}
float calculateSOC(float voltage) {
float soc = (voltage - 3.0) / (4.2 - 3.0) * 100.0;
return constrain(soc, 0, 100);
}
// ---------- LOOP ----------
Output Serial Monitor
Message (Enter to send message to 'ESP32 Dev Module' on 'COM9')
rottle: 255 | PWM: 213 | Curr: 0.14A | Temp: 0.00C | SOC: 28.00% | dI: 0.00 | Stress: 0.18 | Mode: EMERGENCY
rottle: 255 | PWM: 216 | Curr: 0.15A | Temp: 0.00C | SOC: 28.00% | dI: 0.00 | Stress: 0.18 | Mode: EMERGENCY
rottle: 255 | PWM: 210 | Curr: 0.19A | Temp: 0.00C | SOC: 28.00% | dI: 0.05 | Stress: 0.19 | Mode: EMERGENCY
rottle: 255 | PWM: 204 | Curr: 0.19A | Temp: 0.00C | SOC: 28.00% | dI: 0.00 | Stress: 0.18 | Mode: NORMAL
rottle: 255 | PWM: 207 | Curr: 0.20A | Temp: 0.00C | SOC: 27.00% | dI: 0.01 | Stress: 0.18 | Mode: NORMAL
rottle: 255 | PWM: 207 | Curr: 0.20A | Temp: 6.96C | SOC: 27.00% | dI: 0.00 | Stress: 0.19 | Mode: NORMAL
rottle: 255 | PWM: 201 | Curr: 0.22A | Temp: 39.56C | SOC: 27.00% | dI: 0.02 | Stress: 0.26 | Mode: NORMAL
rottle: 255 | PWM: 195 | Curr: 0.21A | Temp: 50.79C | SOC: 27.00% | dI: 0.01 | Stress: 0.34 | Mode: NORMAL
rottle: 255 | PWM: 189 | Curr: 0.21A | Temp: 48.47C | SOC: 27.00% | dI: 0.00 | Stress: 0.32 | Mode: NORMAL
rottle: 255 | PWM: 183 | Curr: 0.20A | Temp: 52.75C | SOC: 27.00% | dI: 0.01 | Stress: 0.35 | Mode: NORMAL
rottle: 255 | PWM: 177 | Curr: 0.18A | Temp: 50.67C | SOC: 27.00% | dI: 0.03 | Stress: 0.34 | Mode: NORMAL
rottle: 255 | PWM: 171 | Curr: 0.14A | Temp: 48.35C | SOC: 27.00% | dI: 0.04 | Stress: 0.32 | Mode: NORMAL
rottle: 255 | PWM: 174 | Curr: 0.13A | Temp: 0.00C | SOC: 27.00% | dI: 0.01 | Stress: 0.19 | Mode: NORMAL
rottle: 255 | PWM: 177 | Curr: 0.13A | Temp: 0.00C | SOC: 27.00% | dI: 0.00 | Stress: 0.18 | Mode: NORMAL
Search


VICTUS

6.2 Performance Analysis



The system demonstrated stable and adaptive behavior under varying conditions:
Stable Output:
Filtering ensured smooth sensor readings, eliminating fluctuations.
Gradual Speed Reduction:
Motor speed reduced in stages instead of abrupt drops, improving control.
Clear Mode Difference:
Normal Mode → noticeable speed reduction under stress
Emergency Mode → maintained higher speed
Battery Protection:
Automatic cutoff at low SOC prevents deep discharge.
Low Temperature Behavior:
At lower temperatures, motor response naturally reduced due to battery inefficiency, and the system handled this smoothly without instability.

6.3 Stress vs PWM Behavior

The relationship between stress and motor speed was observed as:
As stress increases → PWM decreases
The relationship is nonlinear and smooth, not abrupt
Filtering ensures stable transitions
Key Insight:
At low stress → nearly full motor speed
At medium stress → controlled reduction
At high stress → strong limitation
In emergency mode → minimal restriction
X-axis: Stress
Y-axis: PWM

7.Comparative Study with Existing Systems

Modern electric vehicle (EV) control systems employ advanced strategies such as Model Predictive Control (MPC), torque arbitration, and State-of-Power (SOP) estimation to regulate motor performance based on battery conditions. These approaches rely on complex mathematical models and centralized architectures involving Vehicle Control Units (VCU) and Battery Management Systems (BMS).

While such systems provide high accuracy and optimized performance, they are computationally intensive, expensive, and primarily suited for high-end automotive applications. Their implementation in low-cost or embedded platforms is limited due to hardware and processing constraints.

The proposed system addresses these limitations by introducing a simplified yet effective real-time control strategy based on a nonlinear battery stress model implemented on an ESP32 microcontroller. Instead of predictive control, the system focuses on immediate adaptability using real-time sensor inputs.

The comparison between existing systems and the proposed system is summarized below:

Feature| Existing Systems| Proposed System
Control Approach| Lookup tables, MPC, SOP-based control| Nonlinear stress-based real-time control
System Complexity| High (requires advanced computation)| Moderate (embedded implementation)
Hardware Requirement| VCU + BMS (high-end systems)| ESP32-based low-cost architecture
Adaptability| Limited in low-cost setups| High (real-time adaptive response)
Dynamic Behavior Handling| Predictive models| Real-time response using dI/dt
Temperature Handling| Focus on high-temperature protection| Handles both high and low temperature effects
Driver Control| No direct override in basic systems| Emergency mode override available
Torque Transition| Abrupt in simple systems| Smooth PWM-based control

Key Insight:

Existing systems emphasize predictive accuracy and optimization using complex models, whereas the proposed system prioritizes real-time adaptability, simplicity, and practical implementation. By combining nonlinear stress modeling with embedded control, the system achieves a balance between performance, safety, and cost, making it suitable for scalable and low-cost applications.

8. PROJECT TIMELINE

Phase Work	Duration
ResearchStudy EV systems Design	1 week
System planning Implementation	1 week
Coding + hardware	2 week
Testing Debugging	1 week
Documentation and Report writing	1 week

9. Cost Analysis

The proposed system is designed using low-cost and easily available electronic components, making it suitable for educational and prototype-level applications. The approximate cost of each component is listed below based on typical market prices in India.

Total Estimated Cost: ₹1480 (Approx.)

S.No	component	Cost in Rs
1	ESP32 Development Board	300
2	INA219 Current Sensor| 1	150
3	Temperature Sensor (LM35)	50
4	Voltage Sensor Module| 1	80
5	Motor Driver (L298N)	130
6	DC Motor	150
7	Push Button	20
8	Breadboard	100
9	Battery (12V)	300

The overall cost remains under ₹1500, demonstrating that the proposed system can be implemented as a low-cost embedded solution without requiring expensive hardware or complex infrastructure.

10. FUTURE WORK

Integration with full EV systems
AI-based predictive stress control
IoT monitoring dashboard
Advanced SOC estimation (Kalman Filter)
Regenerative braking integration

Lab Test / Simulation Report for Critical Functions:

The proposed system was validated under varying operating conditions including changes in throttle input, load, and temperature. Real-time monitoring of current, voltage, and State of Charge (SOC) was performed to evaluate system behavior.

The system demonstrated adaptive torque regulation, wherein motor PWM output decreased proportionally with increasing battery stress. Mode switching functionality was verified, showing controlled performance reduction in Normal mode and higher torque availability in Emergency mode.

Battery protection mechanisms were validated through SOC-based cutoff, where motor operation was restricted below a predefined threshold. Additionally, stable operation was observed under varying temperature conditions, including low-temperature scenarios.

These results confirm the reliability and effectiveness of the proposed stress-based motor control strategy.

Analytical Model Validation Document

The proposed system utilizes a nonlinear Battery Stress Index (BSI) to represent the internal stress experienced by the battery under varying operating conditions.

The model is defined as:

$$BSI = w_1 (I/I_{max})^2 + w_2 (T/T_{max})^3 + w_3 1\text{-}SOC) + w_4 \, | \, dI/dt \, |$$

The validity of this model was evaluated based on its response to changes in input parameters.

- Current Influence:
  The quadratic term ensures that stress increases rapidly with higher current, reflecting real battery behavior under heavy load conditions.

- Temperature Influence:
  The cubic term emphasizes thermal effects, ensuring that high temperatures significantly impact stress estimation.

- State of Charge (SOC):
  As SOC decreases, the stress value increases, accurately representing reduced battery capability.

- Dynamic Behavior (dI/dt):
  The inclusion of current variation enables the model to capture transient conditions such as sudden acceleration or load changes.

Experimental observations confirmed that the stress value responds logically and

consistently to all input variations. The model effectively represents real-time battery behavior and is suitable for embedded implementation without requiring complex computation.

Defined Performance Metrics List:

To evaluate the effectiveness of the proposed system, the following performance metrics were defined:

- Stress Responsiveness:
  Ability of the system to reflect real-time changes in current, temperature, and SOC.

- PWM Stability:
  Smoothness of motor speed variation without abrupt fluctuations.

- Mode Transition Efficiency:
  Response time and behavior during switching between Normal and Emergency modes.

- Battery Protection Effectiveness:
  Ability to prevent unsafe operating conditions such as low SOC and high stress.

- Thermal Adaptability:
  System performance under varying temperature conditions, including low-temperature operation.

- Dynamic Load Handling:
  Capability to respond to sudden changes in load using dI/dt-based adjustment.

These metrics collectively ensure that the system achieves both safety and performance objectives.

Representative Data Simulation Files:

The system performance was validated using real-time experimental data obtained from hardware implementation.

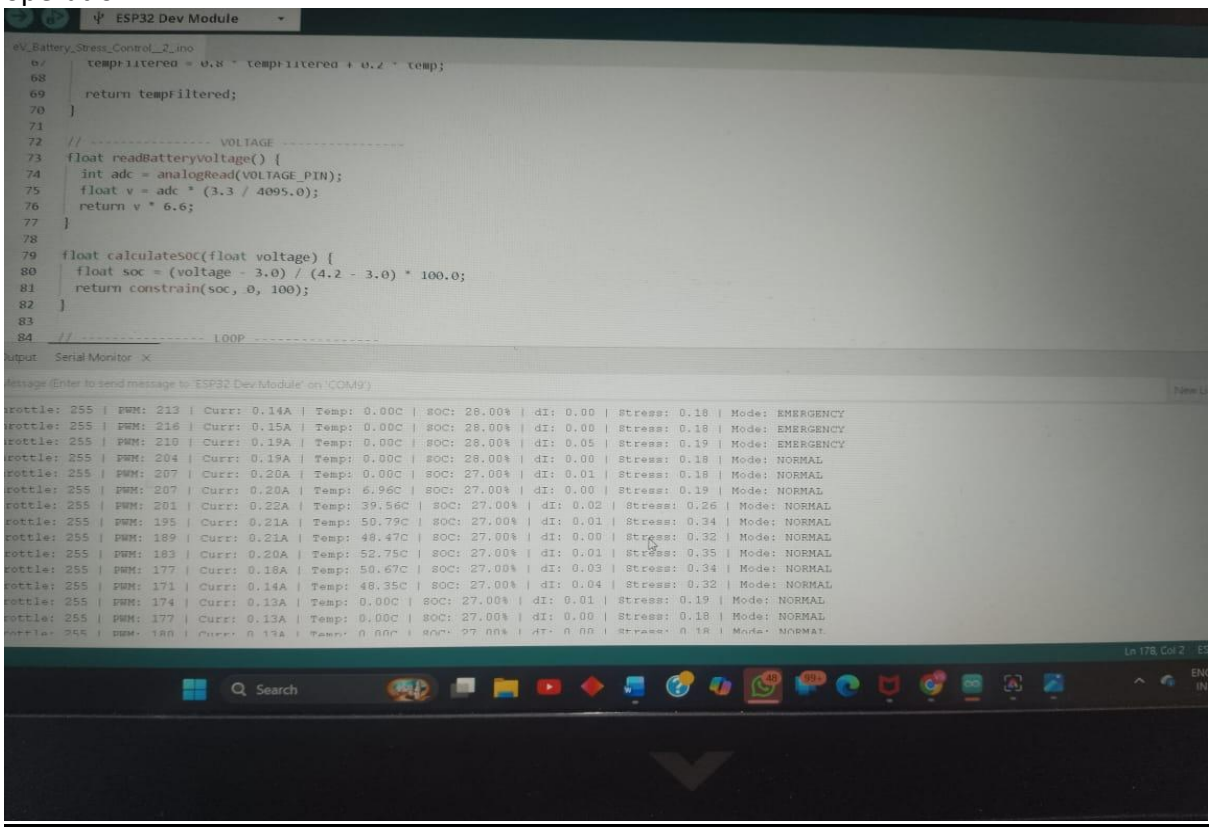
The following parameters were recorded during testing:

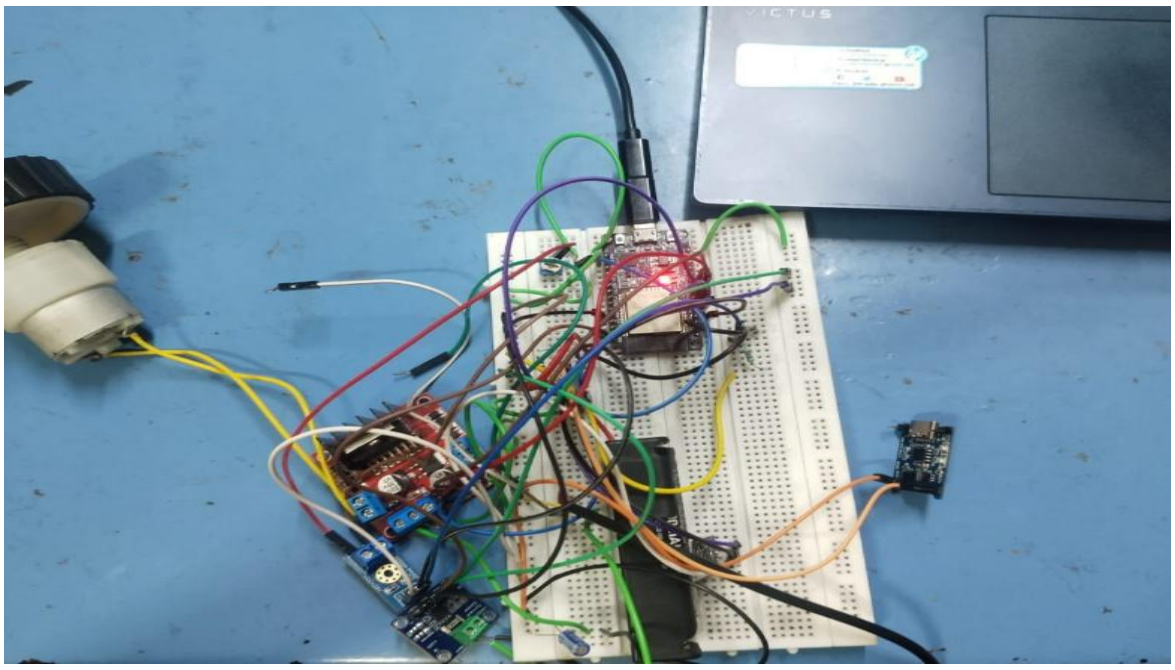
- Throttle input (ADC value mapped to PWM)
- Motor PWM output
- Battery current (INA219 sensor)
- Temperature readings
- Simulated SOC and voltage-based SOC
- Computed stress value

The data demonstrated the following trends:

- Increase in stress resulted in a controlled decrease in PWM output
- Emergency mode maintained higher PWM compared to Normal mode
- Stable output was achieved due to filtering of sensor signals
- Smooth transitions were observed during speed changes

These results validate the effectiveness of the proposed control strategy in real-time operation.





Application Feasibility:

The proposed system demonstrates strong feasibility for real-world applications in electric vehicle motor control and battery management systems.

The design is implemented using an ESP32 microcontroller, making it cost-effective and suitable for embedded applications. The system operates in real time and does not require complex computational resources, unlike advanced predictive control methods.

Key advantages supporting feasibility include:

- Low-cost hardware implementation
- Real-time adaptive control capability
- Minimal computational requirements
- Scalability for integration with existing EV systems
- Compatibility with standard sensors and motor drivers

The system can be effectively deployed in small-scale electric vehicles, prototype systems, and educational platforms, with potential for further development into commercial applications.

Initial Scale-Up Considerations:

For large-scale or commercial deployment, several enhancements can be considered to improve system robustness and performance.

- Replace the ESP32 with an automotive-grade ECU or VCU
- Integrate with a full Battery Management System (BMS) for accurate SOC and thermal data
- Use high-precision sensors for improved measurement accuracy
- Implement CAN communication for vehicle-level integration
- Enhance thermal management using active cooling systems
- Incorporate advanced algorithms such as Model Predictive Control (MPC) for improved optimization

Despite its simplicity, the current system provides a strong foundation for scalable battery-aware motor control and can be extended to meet industrial requirements with minimal modifications.

11.References:

1. United States Patent No. US10427546B2. (2019). *Battery power capability estimation for a vehicle*. U.S. Patent and Trademark Office. Retrieved from https://patents.google.com/patent/US10427546B2/en

2. United States Patent No. US8633663B2. (2014). *Methods and systems for controlling an electric motor based on battery power limits*. U.S. Patent and Trademark Office. Retrieved from https://patents.google.com/patent/US8633663B2/en

3. United States Patent No. US10112613B2. (2018). *Model predictive control for vehicle energy management*. U.S. Patent and Trademark Office. Retrieved from https://patents.google.com/patent/US10112613B2/en

4. United States Patent Application No. US20120139459A1. (2012). *Control device for an electric vehicle*. U.S. Patent and Trademark Office. Retrieved from https://patents.google.com/patent/US20120139459A1/en

5 United States Patent No. US11203273B2. (2021). *Torque arbitration and management for electric vehicles*. U.S. Patent and Trademark Office. Retrieved from https://patents.google.com/patent/US11203273B2/en

6. United States Patent No. US7683573B2. (2010). *Apparatus and method for estimating state of power of a battery*. U.S. Patent and Trademark Office. Retrieved from https://patents.google.com/patent/US7683573B2/en

7 Moura, S. J., Stein, J. L., & Fathy, H. K. (2011). *Battery-constrained optimal control of an electric vehicle powertrain*. IEEE Transactions on Control Systems Technology, 19(6), 1585–1597. https://doi.org/10.1109/TCST.2010.2087023

8 Sun, L., Ma, J., Nie, S., & Huang, Y. (2019). *A novel state-of-power estimation method for lithium-ion batteries in electric vehicles*. IEEE Transactions on Industrial Electronics, 66(10), 7703–7712. https://doi.org/10.1109/TIE.2018.2880705

9 Singh, M. K. G., & Williamson, S. S. (2021). *Comprehensive review of battery management systems (BMS) for electric vehicles*. IEEE Transactions on Power Electronics, 36(12), 14321–14340. https://doi.org/10.1109/TPEL.2021.3082531

12.Appendix:

Code:

```
#include <Wire.h>
#include <Adafruit_INA219.h>
Adafruit_INA219 ina219;
#define ENA 25
#define IN1 26
#define IN2 27
#define THROTTLE_PIN 32
#define TEMP_PIN 34
#define BUTTON_PIN 33
#define VOLTAGE_PIN 36
#define PWM_CHANNEL 0
#define PWM_FREQ 5000
#define PWM_RES 8
bool emergencyMode = false;
bool lastButtonState = HIGH;
float soc_sim = 100.0;
int currentPWM = 0;
unsigned long lastSOCupdate = 0
float tempFiltered = 25;
float currentFiltered = 0;
float stressFiltered = 0;
float prevCurrent = 0;
float Imax = 1.5;
float Tmax = 60.0;
void setup() {
  Serial.begin(115200);
  delay(2000);
  Wire.begin(21, 22);
  if (!ina219.begin()) {
    Serial.println("INA219 ERROR!");
    while (1);
  }

  pinMode(IN1, OUTPUT);
  pinMode(IN2, OUTPUT);
  pinMode(BUTTON_PIN, INPUT_PULLUP);
  digitalWrite(IN1, HIGH);
  digitalWrite(IN2, LOW);
  ledcSetup(PWM_CHANNEL, PWM_FREQ, PWM_RES);
  ledcAttachPin(ENA, PWM_CHANNEL);
  Serial.println("SYSTEM READY");
}
float readTemperature() {
  int adc = analogRead(TEMP_PIN);
  float voltage = adc * (5.0 / 4095.0);
  float temp = voltage * 100.0;
  tempFiltered = 0.85 * tempFiltered + 0.15 * temp;

  return tempFiltered;
}
float readBatteryVoltage() {
  int adc = analogRead(VOLTAGE_PIN);
  float v = adc * (3.3 / 4095.0);
  return v * 6.6;
}
float calculateSOC(float voltage) {
  float soc = (voltage - 3.0) / (4.2 - 3.0) * 100.0;
  return constrain(soc, 0, 100);
void loop() {
  bool buttonState = digitalRead(BUTTON_PIN);
```

```
if (buttonState == LOW && lastButtonState == HIGH) {
  emergencyMode = !emergencyMode;
  delay(200);
}
lastButtonState = buttonState;

int throttleADC = analogRead(THROTTLE_PIN);
int throttlePWM = map(throttleADC, 0, 4095, 0, 255);

float current = ina219.getCurrent_mA() / 1000.0;
currentFiltered = 0.85 * currentFiltered + 0.15 * current;

float displayCurrent = currentFiltered * 10.0;
float dI = abs(currentFiltered - prevCurrent);
prevCurrent = currentFiltered;

float temp = readTemperature()
if (millis() - lastSOCupdate > 5000) {
  soc_sim -= 1;
  if (soc_sim < 0) soc_sim = 0;
  lastSOCupdate = millis();
}

float batteryVoltage = readBatteryVoltage();
float soc_real = calculateSOC(batteryVoltage);

// ---------- NORMALIZATION ----------
float currentNorm = constrain(currentFiltered / Imax, 0, 1);
float tempNorm = constrain(temp / Tmax, 0, 1);
float socNorm = 1.0 - (soc_sim / 100.0);
float rawStress =
    0.3 * pow(currentNorm, 2) +
    0.25 * pow(tempNorm, 3) +
    0.25 * socNorm +
    0.2 * constrain(dI, 0, 1);
stressFiltered = 0.9 * stressFiltered + 0.1 * rawStress;
float stress = constrain(stressFiltered, 0, 1)
float derateFactor = 1.0;
if (currentFiltered > 1.2) derateFactor = 0.6;
else if (currentFiltered > 0.8) derateFactor = 0.75;
else if (currentFiltered > 0.4) derateFactor = 0.9;
int targetPWM;

if (!emergencyMode) {
  // NORMAL MODE (clear reduction)
  targetPWM = throttlePWM * (1.0 - stress) * derateFactor;
}
else {
  targetPWM = throttlePWM * (0.95 - 0.2 * stress);

  if (stress > 0.9) {
    targetPWM *= 0.85; // only extreme case
  }
}
if (soc_sim <= 5) targetPWM = 0;

targetPWM = constrain(targetPWM, 0, 255);

int stepUp = 5;
int stepDown = 8;

if (currentPWM < targetPWM) currentPWM += stepUp;
else if (currentPWM > targetPWM) currentPWM -= stepDown;

currentPWM = constrain(currentPWM, 0, 255);

ledcWrite(PWM_CHANNEL, currentPWM);
Serial.print("Throttle: "); Serial.print(throttlePWM);
Serial.print(" | PWM: "); Serial.print(currentPWM);
Serial.print(" | Curr: "); Serial.print(displayCurrent);
Serial.print(" A | Temp: "); Serial.print(temp);
```

```
  Serial.print(" C | SOC(sim): "); Serial.print(soc_sim);
  Serial.print(" % | Volt: "); Serial.print(batteryVoltage);
  Serial.print(" % | Stress: "); Serial.print(stress);
  Serial.print(" | Mode: ");
  Serial.println(emergencyMode ? "EMERGENCY" : "NORMAL");

  delay(200);
}
```